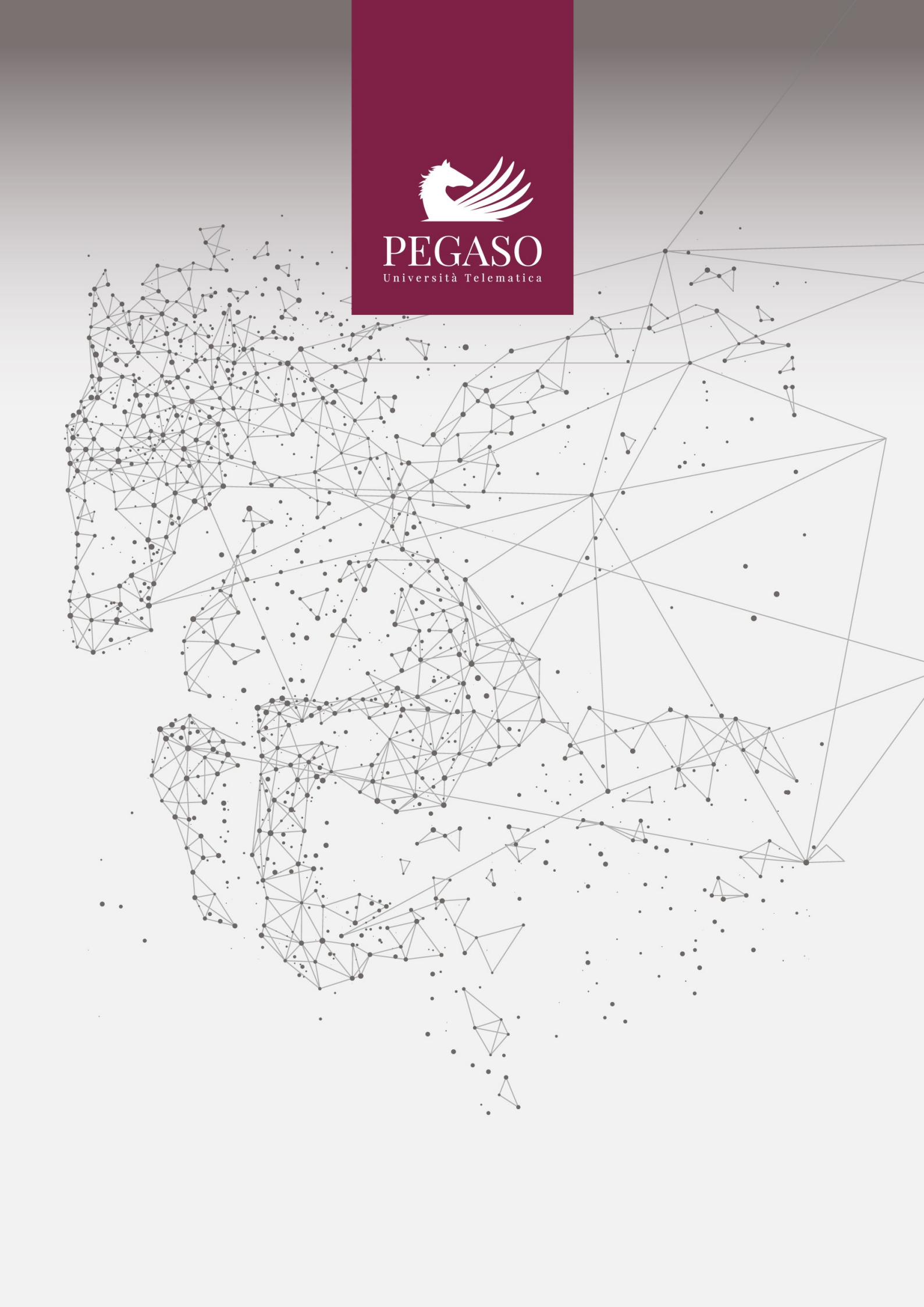

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. ACCESSO AI DATI SUL WEB 4
3. BEAUTIFUL SOUP 7
4. BEAUTIFUL SOUP PER L'ANALISI DELLE TABELLE 11
BIBLIOGRAFIA 19

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presenta una serie di utility per il recupero e l'analisi dei dati delle reti propedeutiche anche per la creazione di grafi. In particolare, vengono illustrati i metodi di gestione degli URL e di recupero del codice HTML delle pagine Web. Viene introdotta anche una libreria Python di estrazione di dati da file HTML e XML, Beautiful Soup, che semplifica il lavoro del programmatore. Infine, vengono illustrate, attraverso un esempio, le potenzialità di Beautiful Soup per l'estrazione di dati da tabelle.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Accesso ai dati sul Web

Al fine di utilizzare i dati delle reti, è necessario introdurre un modulo contenente le funzioni necessarie per il *download dei dati dal web*. Per tale scopo, Python mette a disposizione il modulo **urllib.request**. Tale funzionalità permette di procedere con download dei dati iterativi o ricorsivi automatizzati.

La prima funzione per poter accedere ai dati del Web è **urlopen**(url) che permette di aprire l'URL di una pagina web e di ottenerne l'handle (gestore). Dopo l'apertura, l'handle dell'URL è simile all'handle di un file di sola lettura. Su di esso, per accedere ai dati, infatti, è possibile utilizzare le funzioni *read*(), *readline*() e *readlines*().

A causa della natura dinamica del Web e di Internet, la probabilità di non poter aprire un URL è più elevata rispetto a quella di non poter aprire un file su disco. Pertanto, si ricorda che, ogni volta che si invoca una funzione che opera sul Web, è necessario scrivere il codice per gestire le eccezioni.

Nel codice seguente, dopo l'importazione della libreria urllib.request, si inserisce in un ***blocco try-except*** per la gestione dell'eccezione, l'istruzione di lettura del codice HTML della pagina web https://www.unimercatorum.it che verrà a sua volta copiato nell'oggetto webdoc:

with urlib.request.urlopen("https://www.unimercatorum.it") as webdoc

Successivamente, si procede alla stampa di ciascuna riga del codice HTML contenuta in webdoc. L'except ci permetterà di gestire l'eccezione nei casi in cui non è possibile aprire la pagina web indicata dall'URL.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import urllib.request

try:
    with urllib.request.urlopen("https://www.unimercatorum.it/") as webdoc:
        for riga in webdoc:
            print(riga)    #se la lettura ha successo, la connessione viene chiusa automaticamente


except:
    print("Non è possibile aprire il link web")
```

```
b'<!DOCTYPE html>\r\n'
b'<html lang="it">\r\n'
b'\r\n'
b'\t<head>\r\n'
b'\t\t<meta charset="utf-8">\r\n'
b'\t\t<meta http-equiv="X-UA-Compatible" content="IE=edge">\r\n'
b'\t\t<meta name="viewport" content="width=device-width, initial-scale=1">\r\n'
b'\t\t<title>Universitas Mercatorum | Universit\xc3\xa0 Telematica</title>\r\n'
b'\t\t\t\t\t<meta name="description" content="Entra a far parte delle eccellenze italiane con Unime
\xa0 telematica delle Camere di Commercio. Vieni su unimercatorum.it">\r\n'
b'\t\t\r\n'
b'\t\t<!-- Canonical -->\r\n'
b'\t\t<link rel="canonical" href="https://www.unimercatorum.it/">\r\n'
b'\r\n'
b'\t\t<!-- Preconnect -->\r\n'
b'\t\t<link rel="preconnect" href="https://www.google-analytics.com">\r\n'
b'\t\t<link rel="preconnect" href="https://static.xx.fbcdn.net">\r\n'
b'\t\t<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>\r\n'
b'\r\n'
```

Se il dataset di interesse viene fornito da un sito web che richiede una forma di **autenticazione**, la funzione *urlopen*() non funzionerà. Al suo posto si deve utilizzare un modulo che offre la sicurezza SSL (Secure Sockets Layer) come ad esempio *OpenSSL*.

Il modulo **urlib.parse** fornisce tutti gli strumenti necessari per il *parsing* (analisi) e l'*unparsing* (costruzione) di indirizzi URL. La funzione ***urlparse***() suddivide un URL in una tupla di sei elementi:

1. schema – es. http;
2. indirizzo di rete;
3. percorso nel file system;
4. parametri;
5. query;
6. frammento.

La funzione ***urlunparse****(parts)*, invece, costruisce un URL valido utilizzando le parti restituite da ***urlparse()***. Se si esegue il parsing di un URL e poi l'unparsing delle parti generate, il risultato può essere leggermente diverso dall'URL originale, ma sarà funzionalmente equivalente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nell'esempio riportato nel codice seguente, viene richiamata prima la funzione urlparse() sull'URL:

"https://www.unimercatorum.it/corsi-di-laurea/statistica-e-big-data#obiettivi"

Successivamente, sul risultato della funzione *urlparse*() viene richiamata la funzione *urlunparse*(). In questo caso, il risultato finale è identico all'URL di partenza.

```
import urllib.parse

URL = "https://www.unimercatorum.it/corsi-di-laurea/statistica-e-big-data#obiettivi"
parts = urllib.parse.urlparse(URL)
print(parts)

print(urllib.parse.urlunparse(parts))
```

```
ParseResult(scheme='https', netloc='www.unimercatorum.it', path='/corsi-di-laurea/statistica-e-big-da
ta', params='', query='', fragment='obiettivi')
https://www.unimercatorum.it/corsi-di-laurea/statistica-e-big-data#obiettivi
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Beautiful Soup

Beautiful Soup è una libreria Python per estrarre dati da file HTML e XML (libreria già presente in Anaconda). La libreria permette di individuare in modo automatico i tag HTML presenti nel testo recuperato dal Web.

Per comprendere meglio il suo funzionamento procederemo attraverso i seguenti passi:

- partiamo da un semplice testo HTML (parte della storia «Alice nel Paese delle Meraviglie»);
- analizziamo il testo attraverso BeautifulSoup;
- accediamo ai tag del testo HTML e al testo della storia.

La sorgente del file HTML è la seguente:

```
<html><head><title>The Dormouse's story</title></head>
<body>
<p class="title"><b>The Dormouse's story</b></p>

<p class="story">Once upon a time there were three little sisters; and their names were
<a href="http://example.com/elsie" class="sister" id="link1">Elsie</a>,
<a href="http://example.com/lacie" class="sister" id="link2">Lacie</a> and
<a href="http://example.com/tillie" class="sister" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>

<p class="story">...</p>
```

Tale codice HTML aperto con un browser, mostra la seguente pagina. Si notino i tre link di esempio "Elsie", "Lacie" and "Tillie" indicati con il tag <a> nel codice sorgente.

The Dormouse's story

Once upon a time there were three little sisters; and their names were Elsie, Lacie and Tillie; and they lived at the bottom of a well.

...

Procediamo con l'analisi attraverso Beautiful Soup. In primo luogo, è necessario importare la libreria Beautiful Soup nel seguente modo:

from bs4 import BeautifulSoup

Successivamente, riportiamo il codice sorgente HTML visto in precedenza in una stringa (nel nostro caso *html_doc*). Questo passaggio è effettuato solo per semplicità. Ovviamente, è possibile ricavare il codice HTML direttamente da una pagina web come visto in precedenza con la funzione *urlopen*().

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Infine, invochiamo la funzione *BeautifulSoup*() passandole due argomenti:

1. il codice sorgente HTML;
2. una stringa che indica che tipo di parser (analizzatore) utilizzare. Nel nostro caso sarà un parser HTML, quindi passeremo '*html.parser*'.

Per visualizzare il risultato, viene visualizzato a schermo il risultato del metodo *prettify*() richiamato sull'oggetto soup. La funzione *prettify*() permette di visualizzare come i tag HTML sono annidati nel documento.

```
from bs4 import BeautifulSoup

html_doc="""<html><head><title>The Dormouse's story</title></head>
<body>
<p class="title"><b>The Dormouse's story</b></p>

<p class="story">Once upon a time there were three little sisters; and their names were
<a href="http://example.com/elsie" class="sister" id="link1">Elsie</a>,
<a href="http://example.com/lacie" class="sister" id="link2">Lacie</a> and
<a href="http://example.com/tillie" class="sister" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>

<p class="story">...</p>"""


soup = BeautifulSoup(html_doc, 'html.parser')

print(soup.prettify())
```

Il risultato del codice precedente viene riportato di seguito.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
<html>
 <head>
  <title>
   The Dormouse's story
  </title>
 </head>
 <body>
  <p class="title">
   <b>
    The Dormouse's story
   </b>
  </p>
  <p class="story">
   Once upon a time there were three little sisters; and their names were
   <a class="sister" href="http://example.com/elsie" id="link1">
    Elsie
   </a>
   ,
   <a class="sister" href="http://example.com/lacie" id="link2">
    Lacie
   </a>
   and
   <a class="sister" href="http://example.com/tillie" id="link3">
    Tillie
   </a>
   ;
and they lived at the bottom of a well.
  </p>
  <p class="story">
   ...
  </p>
 </body>
</html>
```

Per accedere al solo testo del contenuto dell'HTML (quindi esclusi i tag) è possibile richiamare il metodo ***get_text***() come riportato nel caso seguente.

```
print(soup.get_text())
```

```
The Dormouse's story

The Dormouse's story
Once upon a time there were three little sisters; and their names were
Elsie,
Lacie and
Tillie;
and they lived at the bottom of a well.
...
```

Per navigare nella struttura dati restituita dopo l'analisi eseguita con Beautiful Soup, è possibile utilizzare proprietà e metodi tra i quali:

- **title** (proprietà) per estrarre il tag <title>;
- **title.name** (proprietà) per estrarre nome del tag <title>;
- **title.name.string** (proprietà) per estrarre il contenuto tra i tag <title> e <\title>;

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **p** (proprietà) per estrarre il primo paragrafo con tag <p>;
- a (proprietà) per estrarre il primo link con tag <a>;
- **findall**(t) (metodo) per estrarre tutti i tag indicati da t;
- **find**(id) (metodo) per estrarre tutti i tag con identificativo indicato da id.

Nel codice seguente, si riportano alcuni esempi di applicazione dei metodi appena citati.

Per comodità di lettura, il risultato di ciascuna proprietà/metodo viene riportato come commento subito dopo l'istruzione.

```
soup.title
# <title>The Dormouse's story</title>

soup.title.name
# u'title'

soup.title.string
# u'The Dormouse's story'

soup.title.parent.name
# u'head'

soup.p
# <p class="title"><b>The Dormouse's story</b></p>

soup.p['class']
# u'title'

soup.a
# <a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>

soup.find_all('a')
# [<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
#  <a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>,
#  <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>]

soup.find(id="link3")
# <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Beautiful Soup per l'analisi delle tabelle

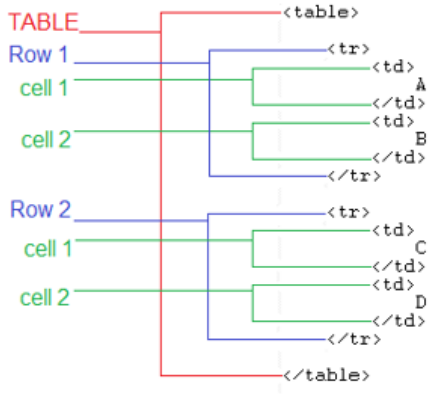
Molto interessante è anche la capacità di Beautiful Soup di trovare i **tag <table>** che contengono **dati in forma tabellare.**

Prima di procedere, ricordiamo brevemente la struttura (inclusi i tag) di una tabella in HTML. Ogni tabella comincia con il tag <table> e termina con il tag di chiusura </table>. All'interno dei tag <table>, secondo una struttura nidificata e gerarchica di tag, per ogni riga della tabella, è necessario utilizzare i tag di apertura e chiusura <tr> e </tr>, dove tr sta proprio per *table row*.

Infine, per ciascuna cella della tabella all'interno di una riga (e, quindi, tra i tag di riga) sono necessari i tag di apertura e chiusura <td> e </td> (td sta, infatti, per *table data*).

Nella figura seguente, si riporta uno schema logico intuitivo (a sinistra) e la struttura gerarchica di una tabella 2x2 con l'indicazione esplicita di tabella, righe e celle (a destra).

	<td>	<td>	<td>
<tr>	Content	Content	Content
<tr>	Content	Content	Content
<tr>	Content	Content	Content



Un esempio semplice, ma chiaro di una tabella in HTML è il seguente che mostra una tabella 4x2. Si osservi che i valori contenuti nelle celle permettono di comprendere meglio la struttura della tabella stessa.

```
<table>
<tr><td>Colonna 1</td><td>Colonna 2</td></tr>
<tr><td>Dato 1,1</td><td>Dato 1,2</td></tr>
<tr><td>Dato 2,1</td><td>Dato 2,2</td></tr>
<tr><td>Dato 3,1</td><td>Dato 3,2</td></tr>
</table>
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Colonna 1	Colonna 2
Dato 1,1	Dato 1,2
Dato 2,1	Dato 2,2
Dato 3,1	Dato 3,2

Una tabella di esempio molto interessante ai fini dell'elaborazione dei dati si trova alla pagina di Wikipedia all'indirizzo: https://en.wikipedia.org/wiki/List_of_countries_and_territories_by_land_borders

In tale tabella è presente la lista degli Stati e dei territori con indicazione del territorio confinante come mostrato nello screenshot seguente.



Land borders [edit]

Contents: Top · A · B · C · D · E · F · G · H · I · J · K · L · M · N · O · P · Q · R · S · T · U · V · W · X · Y · Z

Country or territory [show]	Total length of land borders		No. of distinct land borders	No. of distinct land neighbours	Land border neighbours and border length [show]
	km[3]	mi			
Abkhazia	382	237	2	2	Russia: 241 km (150 mi) Georgia: 141 km (88 mi)
Afghanistan	5,529	3,436	6	6	People's Republic of China: 76 km (47 mi) Iran: 935 km (581 mi) Pakistan: 2,430 km (1,510 mi) Tajikistan: 1,206 km (749 mi) Turkmenistan: 744 km (462 mi) Uzbekistan: 137 km (85 mi).
Albania	720	450	4	4	Greece: 282 km (175 mi) Kosovo:[4] 112 km (70 mi) North Macedonia: 151 km (94 mi) Montenegro: 172 km (107 mi)

Dall'analisi della tabella precedente è semplice notare che:

1. nella prima colonna c'è l'indicazione di uno Stato;
2. nella seconda colonna sono indicati i chilometri totali del confine dello Stato;
3. nella sesta colonna c'è l'indicazione degli Stati confinanti.

Procediamo, quindi, in modo incrementale all'analisi della tabella mediante Beautiful Soup.

Come primo passo, recuperiamo il codice HTML della pagina indicata e cerchiamo la prima (si noti l'indice *[0]*) tabella ("*table*") di classe "*wikitable sortable*" *({"class": "wikitable sortable"})*.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import bs4 as BeautifulSoup
import urllib.request

URL = "https://en.wikipedia.org/wiki/List_of_countries_and_territories_by_land_borders"
with urllib.request.urlopen(URL) as source:
    soup = BeautifulSoup.BeautifulSoup(source)

table = soup.findAll("table", {"class": "wikitable sortable"})[0]

print(table)
```

```
<table class="wikitable sortable">
<tbody><tr>
<th align="left" rowspan="2" scope="col" valign="top" width="150pt"><div class="mw-collapsible mw-collapsed" style="box-sizin
g:border-box;width:100%;font-size:95%;padding:4px;border:none;"><div style="font-size:100%;line-height:1.6;font-weight:bold;b
ackground:transparent;text-align:center;;height:auto;"><div style="margin-right:4em;line-height:125%;height:auto;">Country or
territory</div></div><div class="mw-collapsible-content" style="background:transparent;text-align:left;;">
<center><small><i>(Territories without full sovereignty in italics)</i> <sup class="reference" id="cite_ref-sovereign_2-0"><a
href="#cite_note-sovereign-2">[2]</a></sup></small></center></div></div>
</th>
<th colspan="2" scope="col" valign="top">Total length of land borders
</th>
<th align="left" data-sort-type="number" rowspan="2" scope="col" valign="top" width="80pt"><abbr title="Number">No.</abbr> of
distinct land borders
```

Il secondo passo è quello di estrarre dal codice HTML di table tutte le righe attraverso il metodo *findAll("tr")*. Successivamente, per ogni riga, attraverso il metodo *findAll("td")* si accede a tutte le celle della riga stessa.

```
import bs4 as BeautifulSoup
import urllib.request

URL = "https://en.wikipedia.org/wiki/List_of_countries_and_territories_by_land_borders"
with urllib.request.urlopen(URL) as source:
    soup = BeautifulSoup.BeautifulSoup(source)

table = soup.findAll("table", {"class": "wikitable sortable"})[0]

for row in table.findAll("tr"):
    row = row.findAll('td')
    print(row)
```

```
[]
[]
[<td><i><b><span class="flagicon"><img alt="" class="thumbborder" data-file-height="300" data-file-width="600" decoding="asyn
c" height="12" src="//upload.wikimedia.org/wikipedia/commons/thumb/7/7a/Flag_of_the_Republic_of_Abkhazia.svg/23px-Flag_of_the
_Republic_of_Abkhazia.svg.png" srcset="//upload.wikimedia.org/wikipedia/commons/thumb/7/7a/Flag_of_the_Republic_of_Abkhazia.s
vg/35px-Flag_of_the_Republic_of_Abkhazia.svg.png 1.5x, //upload.wikimedia.org/wikipedia/commons/thumb/7/7a/Flag_of_the_Republ
ic_of_Abkhazia.svg/46px-Flag_of_the_Republic_of_Abkhazia.svg.png 2x" width="23"/> </span><a href="/wiki/Abkhazia" title="Abkh
azia">Abkhazia</a></b></i>
</td>, <td data-sort-value="7005382000000000000" style="text-align:right;">382
</td>, <td data-sort-value="7005382000000000000" style="text-align:right;">237
</td>, <td>2
</td>, <td>2
```

Si effettua, dunque, un controllo per verificare che la riga della tabella abbia dei contenuti (*if row:*). Si estrae, poi, lo Stato contenuto nella prima colonna della tabella attraverso il codice:

list(row[0].strings)[1]

In particolare, vengono estratte tutte le stringhe (*strings*) della prima colonna (*row[0]*) che vengono inserite in una lista attraverso la funzione di Python *list*(). Successivamente, si prende il secondo elemento poiché, come facilmente riscontrabile, la prima stringa risulta essere vuota.

L'intero codice e il risultato generato sono riportati nella figura seguente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import bs4 as BeautifulSoup
import urllib.request

URL = "https://en.wikipedia.org/wiki/List_of_countries_and_territories_by_land_borders"
with urllib.request.urlopen(URL) as source:
    soup = BeautifulSoup.BeautifulSoup(source)

table = soup.findAll("table", {"class": "wikitable sortable"})[0]

for row in table.findAll("tr"):
    row = row.findAll('td')
    if row:
        countryA = list(row[0].strings)[1]
        print(countryA)
```

```
Abkhazia
Afghanistan
Albania
Algeria
Andorra
Angola
Antigua and Barbuda
Argentina
Armenia
Australia
Austria
Azerbaijan
```

Estratto lo Stato dalla prima colonna (*countryA*), si procede con l'estrazione degli Stati confinanti contenuti nella sesta colonna. Si cercano, quindi, tutti i tag '*a*' contenuti nella cella poiché gli Stati confinanti sono dei link. Per ciascuno di essi, dopo il controllo che non ci siano parentesi quadre nel testo, si procede all'estrazione come indicato nel codice seguente:

```
for row in table.findAll("tr"):
    row = row.findAll('td')
    if row:
        countryA = list(row[0].strings)[1]
        print(countryA+" --> ", end="")
        B = row[5].findAll('a')
        for x in B:
            if ("[" not in x.string):
                print(x.string+"\t",end="")
        print()
```

```
Abkhazia --> Russia     Georgia
Afghanistan --> People's Republic of China      Iran    Pakistan        Tajikistan      Turkmenistan    Uzbekistan
Albania --> Greece      Kosovo  North Macedonia Montenegro
Algeria --> Libya       Mali    Mauritania      Morocco Niger   Tunisia Western Sahara
Andorra --> France      Spain
Angola --> Democratic Republic of the Congo     Republic of the Congo   Namibia Zambia
Antigua and Barbuda -->
Argentina --> Bolivia   Brazil  Chile   Paraguay        Uruguay
Armenia --> Azerbaijan  Georgia Iran    Turkey
Australia -->
Austria --> Czech Republic      Germany Hungary Italy   Liechtenstein   Slovakia        Slovenia        Switzerland
```

Si può procedere salvando gli Stati confinanti in una lista e generando un dizionario in cui la **chiave** è *countryA* e il suo **valore** corrispondente è la *lista* appena generata.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
d = dict()

for row in table.findAll("tr")[1:]:
    row = row.findAll('td')
    if row:
        countryA = list(row[0].strings)[1]
        B = row[5].findAll('a')
        countryB_list=list()
        for x in B:
            if ("[" not in x.string):
                countryB_list.append(x.string)
        d[countryA]=countryB_list
print(d)
```

```
{'Abkhazia': ['Russia', 'Georgia'], 'Afghanistan': ["People's Republic of China", 'Iran', 'Pakistan', 'Tajikistan', 'Turkmeni
stan', 'Uzbekistan'], 'Albania': ['Greece', 'Kosovo', 'North Macedonia', 'Montenegro'], 'Algeria': ['Libya', 'Mali', 'Maurita
nia', 'Morocco', 'Niger', 'Tunisia', 'Western Sahara'], 'Andorra': ['France', 'Spain'], 'Angola': ['Democratic Republic of th
e Congo', 'Republic of the Congo', 'Namibia', 'Zambia'], 'Antigua and Barbuda': [], 'Argentina': ['Bolivia', 'Brazil', 'Chil
e', 'Paraguay', 'Uruguay'], 'Armenia': ['Azerbaijan', 'Georgia', 'Iran', 'Turkey'], 'Australia': [], 'Austria': ['Czech Repub
lic', 'Germany', 'Hungary', 'Italy', 'Liechtenstein', 'Slovakia', 'Slovenia', 'Switzerland'], 'Azerbaijan': ['Armenia', 'Geor
gia', 'Iran', 'Russia', 'Turkey'], 'Bahamas': [], 'Bahrain': [], 'Bangladesh': ['India', 'Dahagram-Angarpota', 'Myanmar'], 'B
arbados': [], 'Belarus': ['Latvia', 'Lithuania', 'Poland', 'Russia', 'Ukraine'], 'Belgium': ['France', 'Germany', 'Luxembour
g', 'Netherlands'], 'Belize': ['Guatemala', 'Mexico'], 'Benin': ['Burkina Faso', 'Niger', 'Nigeria', 'Togo'], 'Bhutan': ["Peo
```

È possibile, inoltre, creare una *Series* attraverso la funzione Series di Pandas a cui viene passato il dizionario appena creato. Il risultato sarà, quindi, una Series in cui le etichette saranno gli Stati e i valori corrispondenti saranno gli Stati confinanti contenuti in una lista.

```
d = dict()

for row in table.findAll("tr")[1:]:
    row = row.findAll('td')
    if row:
        countryA = list(row[0].strings)[1]
        B = row[5].findAll('a')
        countryB_list=list()
        for x in B:
            if ("[" not in x.string):
                countryB_list.append(x.string)
        d[countryA]=countryB_list

s = pd.Series(d)
print(s)
```

```
Abkhazia                                        [Russia, Georgia]
Afghanistan       [People's Republic of China, Iran, Pakistan, T...
Albania               [Greece, Kosovo, North Macedonia, Montenegro]
Algeria           [Libya, Mali, Mauritania, Morocco, Niger, Tuni...
Andorra                                          [France, Spain]
                                                 ...
Vietnam                [Cambodia, People's Republic of China, Laos]
Western Sahara                        [Algeria, Mauritania, Morocco]
Yemen                                            [Oman, Saudi Arabia]
Zambia            [Angola, Botswana, Democratic Republic of the ...
Zimbabwe               [Botswana, Mozambique, South Africa, Zambia]
Length: 203, dtype: object
```

Sulla falsariga di quanto fatto con gli Stati confinanti, è possibile estrarre anche il valore numero contenuto nella seconda colonna della tabella (chilometri di confine di ciascuno Stato).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo caso, estrarremo il testo del secondo elemento della riga di celle ([1]) eliminando le virgole di separazione delle migliaia ed eseguendo un casting a float. Si crea, dunque, un altro dizionario *dl*.

```
dc = dict()
dl = dict()

for row in table.findAll("tr")[1:]:
    row = row.findAll('td')
    if row:
        countryA = list(row[0].strings)[1]
        try:
            length = float(row[1].text.replace(",",""))
        except:
            length = 0.0
        B = row[5].findAll('a')
        countryB_list=list()
        for x in B:
            if ("[" not in x.string):
                countryB_list.append(x.string)
        dc[countryA]=countryB_list
        dl[countryA]=length
print(dl)
```

```
{'Abkhazia': 382.0, 'Afghanistan': 5529.0, 'Albania': 720.0, 'Algeria': 6470.0, 'Andorra': 120.0, 'Angola': 5198.0, 'Antigua an
d Barbuda': 0.0, 'Argentina': 9665.0, 'Armenia': 1254.0, 'Australia': 0.0, 'Austria': 2562.0, 'Azerbaijan': 2013.0, 'Bahamas':
0.0, 'Bahrain': 0.0, 'Bangladesh': 4246.0, 'Barbados': 0.0, 'Belarus': 2900.0, 'Belgium': 1385.0, 'Belize': 516.0, 'Benin': 198
9.0, 'Bhutan': 1075.0, 'Bolivia': 6743.0, 'Bosnia and Herzegovina': 1459.0, 'Botswana': 4015.0, 'Brazil': 14691.0, 'Brunei': 38
```

Avendo a disposizione le due Series, sarà possibile ottenere un DataFrame in cui le etichette saranno gli spazi e le due colonne saranno la lunghezza del confine e l'elenco dei paesi confinanti.

```
sc = pd.Series(dc)
sl = pd.Series(dl)
df = pd.DataFrame({'Lunghezza confine':sl, 'Paesi confinanti':sc})
print(df)
```

```
              Lunghezza confine  \
Abkhazia                  382.0
Afghanistan              5529.0
Albania                   720.0
Algeria                  6470.0
Andorra                   120.0
...                         ...
Vietnam                  4639.0
Western Sahara           2046.0
Yemen                    1746.0
Zambia                   5667.0
Zimbabwe                 3066.0

                                             Paesi confinanti
Abkhazia                                       [Russia, Georgia]
Afghanistan     [People's Republic of China, Iran, Pakistan, T...
Albania            [Greece, Kosovo, North Macedonia, Montenegro]
Algeria         [Libya, Mali, Mauritania, Morocco, Niger, Tuni...
Andorra                                          [France, Spain]
...                                                          ...
Vietnam              [Cambodia, People's Republic of China, Laos]
Western Sahara                    [Algeria, Mauritania, Morocco]
Yemen                                        [Oman, Saudi Arabia]
Zambia          [Angola, Botswana, Democratic Republic of the ...
Zimbabwe             [Botswana, Mozambique, South Africa, Zambia]

[203 rows x 2 columns]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sarà così possibile utilizzare tutti i metodi visti per la Data Science sui DataFrame. Essendo la sola colonna «Lunghezza confine» di tipo numerico, il metodo **describe**() richiamato sul DataFrame riporterà le statistiche relative solo a tale colonna.

```
sc = pd.Series(dc)
sl = pd.Series(dl)
df = pd.DataFrame({'Lunghezza confine':sl, 'Paesi confinanti':sc})

print("Statistiche lunghezza confine")
print(df.describe())
```

```
Statistiche lunghezza confine
       Lunghezza confine
count         203.000000
mean         2507.846995
std          3260.988503
min             0.000000
25%           233.000000
50%          1564.000000
75%          3693.500000
max         22147.000000
```

In realtà, anche per l'altra colonna dei Paesi confinanti è possibile calcolare delle statistiche. Prima, però, è necessario eseguire alcune elaborazioni. La prima è quella di eseguire un **map** con una funzione **lambda** che permette di ottenere una Series con il numero di Paesi confinanti per ogni Stato.

```
s = df['Paesi confinanti'].map(lambda x: len(x))
```

```
Abkhazia          2
Afghanistan       6
Albania           4
Algeria           7
Andorra           2
                 ..
Vietnam           3
Western Sahara    3
Yemen             2
Zambia            8
Zimbabwe          4
Name: Paesi confinanti, Length: 203, dtype: int64
```

Richiamando il metodo **describe**() sulla Series ottenuta è possibile conoscere le statistiche sul numero dei Paesi confinanti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
s = df['Paesi confinanti'].map(lambda x: len(x))
print("Statistiche sul numero di Paesi confinanti")
print(s.describe())
```

```
Statistiche sul numero di Paesi confinanti
count    203.000000
mean       3.325123
std        2.848086
min        0.000000
25%        1.000000
50%        3.000000
75%        5.000000
max       18.000000
Name: Paesi confinanti, dtype: float64
```

Dai risultati del metodo **describe**() risulta che il **massimo** numero di Paesi confinanti è **18**.

Attraverso una semplice **map** con una **funzione lambda** è possibile ottenere il filtro da applicare alla Series per conoscere tale Stato in questione.

```
s = df['Paesi confinanti'].map(lambda x: len(x)==18)
print(df[s])
```

```
                          Lunghezza confine  \
People's Republic of China           22147.0

                                                          Paesi confinanti
People's Republic of China  [Afghanistan, Bhutan, Hong Kong, SAR of China,...
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev D. (2017). Data science con Python. Apogeo.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).